CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Muhammet Emin Toraman
Öğrenci numarası (SID)	5001230008
E-posta	m.toraman@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G3
Seçilen ISA ve sürümü (ör. RV32IM)	ARM AArch64
Derleyici ve sürümü	gcc (Debian 14.2.0-19) 14.2.0
Simülatör / araçlar	GNU gdb (Debian 16.3-1) 16.3, GNU objdump (GNU Binutils for Debian) 2.44
Git deposu adresi	https://github.com/th-Emin/CMPE3209-G3-AArch64

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001230008 mod 10	8
D2	SID'in son iki hanesi	5001230008 mod 100	08
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+0+8	19
N	24 + (D2 mod 25)	24 + (8 mod 25)	32

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	256 B / 8 B / 2-way set-assoc. / 32 / 16	256/8/2-way/32/16
Write policy	d çift → write-back, d tek → write-through	d = 8	write-back
k	(S mod 7) + 3	(19 mod 7) + 3	8
Interrupt türü	S çift → Timer, S tek → I/O	19	I/O
f	1.0 + 0.1 × d GHz (varsayım)	1 + 0.1 * 8	1.8 GHz
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	1 cycle ; (20 + 5 * (8 mod 4)) cycle	T1 = 1, T2 = 20
Nc	2 + (S mod 7) (varsayım)	2 + (19 mod 7)	7
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	K = 5001230008 mod 256	184 = 0xB8

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	3	$\log_2$(line bouyutu) = $\log_2$(8)
Line (r) veya set (d)	4	$\log_2$(set sayısı) = $\log_2$(16)
Tag (s − r veya s − d)	25	32 - (set + word) = 32 - (4 + 3)
Toplam	32	3 + 4 + 25

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☒ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> *G3 görevi C99 standardında implemente edilmiştir. Girdi verisi x_0 = SID başlangıç değeriyle LCG algoritması üzerinden deterministik olarak üretilmiş ve global arr dizisine yerleştirilmiştir. İlk geçişte dizi elemanları K = SID mod 256 anahtarı ile XOR'lanarak yerinde şifrelenmiş ve 32-bit sola dairesel kaydırma (rotl) fonksiyonu ile checksum hesaplanmıştır. İkinci geçişte dizi aynı anahtarla tekrar XOR'lanarak yerinde çözülmüş, anlık üretilen LCG değerleriyle kıyaslanıp doğruluk durumu global decrypt_ok değişkenine aktarılmıştır*

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
1.  /* Muhammet Emin Toraman, 5001230008, G3 */
2.  #define N 32 // N'yi global arr tanımlayabilmek için variable olarak tanımlanmadı
3.  const unsigned int SID = 1230008; // 5001230008 2^32'yi aştığı için son 9 hanesi
4.  const unsigned int R = 26;
5.  const unsigned int B = 65;
6.  const unsigned int K = 184;
7.
8.  //Dizi
9.  unsigned int arr[N];
10.
11. //Çıktı
12. unsigned int checksum = 0;
13. char decrypt_ok = 1;
14.
15. unsigned int rotl(unsigned int value, unsigned int shift){
16.   return (value << shift) | (value >> (sizeof(value) * 8 - shift));
17. }
18.
19. int main() {
20.   //Dizi üretme
21.   unsigned int x = SID;
22.   unsigned int i = 0;
23.   while(i < N) {
24.     x = 1664525 * x + 1013904223;
25.     arr[i++] = ((x >> 16) % R) + B;
26.   }
27.
28.   //Diziyi K anahtarıyla şifreleme
29.   i = 0;
30.   while(i < N) {
31.     arr[i] = arr[i] ^ K;
32.     i++;
33.   }
34.
35.   //Checksum Hesabı
36.   i = 0;
37.   while(i < N) {
38.     checksum = rotl(checksum, 1) + arr[i++];
39.   }
40.
41.   //Şifreyi çözeme
42.   x = SID;
43.   i = 0;
44.   while(i < N){
45.     arr[i] = arr[i] ^ K;
46.
47.     x = 1664525 * x + 1013904223;
48.     if(arr[i] != (((x >> 16) % R) + B)) decrypt_ok = 0;
49.     i++;
```

```
50.   }
51.
52.   return 0;
53. }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	9, 25, 31, 45	Global arr tanımlanmış; 1. geçişte şifreleme (s. 31), 2. geçişte şifre çözme (s. 45) yapılmıştır.
Y2	Koşullu dallanma	48	if(arr[i] != ...) satırında çözülen verinin doğruluğu kontrol edilerek koşullu branch sağlanmıştır
Y3	Parametreli, değer döndüren fonksiyon çağrısı	15, 38	Tanımlanan rotl fonksiyonu (s. 15) rotl(checksum, 1) şeklinde (s. 38) çağrılmıştır
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	16, 24, 25, 31, 38	rotl içinde <<, >>, |; LCG'de *, +, %; şifrelemede ^ (XOR) ve checksum'da + kullanılmıştır
Y5	Dizi elemanına store	25, 31, 45	arr[...] = ... atamaları ile dizi elemanlarına bellek yazması gerçekleştirilmiştir
Y6	const int SID ve hesapta kullanımı	3, 21, 42	const unsigned int SID tanımlanmış; LCG başlangıcında x = SID atamalarında kullanılmıştır
Y7	Sonucun tutulduğu global değişken(ler)	12, 13, 38, 48	Çıktı sonuçları global checksum (s. 38) ve decrypt_ok (s. 48) değişkenlerinde saklanmıştır

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
checksum	0x5555555700c0	0x5a4fedd3	/screenshots/outputs.png
decrypt_ok	0x555555570030	1	/screenshots/outputs.png

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı0x86c - 0x8c0	37 - 39
Başlangıç ve bitiş adresi (hex)	0x874 - 0x8c0
Dinamik instruction sayısı	17
Hangi döngü iterasyonu (ör. i = 3)	i = 1

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text			
.data / .bss			

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı			
Load sayısı			
Store sayısı			
Kullanılan register sayısı			

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							

Zorunlu instruction'ların tablodaki sıra numarası:

- ☐ SID'i yükleyen instruction(lar): # ____
- ☐ Koşullu branch: # ____
- ☐ Load: # ____
- ☐ Store: # ____
- ☐ Fonksiyon çağrısı: # ____

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										
16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	

Kernel toplamları	Sayı
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 `ADD B,A` ile karşılaştırarak):

>

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			
MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`.

Ölçü	Değer
Kullanılan derleme (`-O0` / `-O1`)	
Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			
Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		
T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	$Ic \times CPI \times \tau$		
MIPS rate	$f / (CPI \times 10^6)$		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	$Ic \times [p + (m \times k)] \times \tau$		
T, cache var (k = T_s)	$Ic \times [p + (m \times k)] \times \tau$		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	
Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = $1 / [(1 - f) + f / Nc]$	

Alan	Değer
Nc → ∞ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Google Gemini Flash 3.6	0	Tablolardaki hesaplarının doğruluğunu kontrol ettirmek	Tablodaki hesaplar doğru mu, kontrol et.	Yapay zekanın çıkardığı hesap adımlarını, kendi yaptığım hesaplamalarla karşılaştırdım.
Google Gemini Flash 3.6	1	Rapor metninin akıcılığını artırmak ve anlatımı düzenlemek	Yazdığım rapordaki metninleri anlamı bozmadan daha düzgün ve akıcı hale getir.	Düzenlenen metinleri baştan sona okuyup orijinali ile karşılaştırdım, istediğim kavramlardan sapmadığını ve anlam kaybı olmadığını kontrol ettim
Google Gemini Flash 3.6	1.5	İlgili araçların kullanımını öğrenme	GDB ile bellek adresi ve değişken değerini nasıl görürüm?	Önerilen komutlarını doğrudan kendi cihazımdaki çalıştırarak çıktıların doğru şekilde alındığını doğruladım

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza